Informe: Diseño de la base de datos

Gestión para la generación automática de horarios docentes

Bases de Datos II – Ingeniería de Software
Curso 2026–2027

1. Integrantes del equipo

Nombre	**Grupo**	**Usuario**
Ramón Hernández Camacho	C311	`@ramonconda`
Raciel Pupo Santos	C312	`@Rac_7`
Leandro Márquez Blanco	C311	`@usuario_no7`
Charly Blanco Sánchez	C311	`@Charly_BS`
Marlon R. Cuellar García	C312	`@mrcuellar`

Equipo: LA ESCALONETA

Índice

1. Integrantes del equipo **1**

2. Requerimientos Funcionales **4**

2.1. Gestión de personas y roles 4

2.2. Gestión de programas académicos y asignaturas 4

2.3. Gestión de bloques de contenido 4

2.4. Generación y programación de horarios 5

2.5. Revisión y validación de horarios 5

2.6. Asignación y consulta de horarios 6

2.7. Gestión de asistencia 6

2.8. Gestión de solicitudes de cambio de horario 6

2.9. Consultas y reportes 7

2.10. Integración con el almacén de datos histórico 8

3. Requerimientos Informacionales **9**

3.1. Gestión de personas y roles 9

3.2. Programas académicos y asignaturas 9

3.3. Bloques de contenido 9

3.4. Horarios y programación 9

3.5. Revisión y validación 10

3.6. Asignación y consulta de horarios 10

3.7. Asistencia 11

3.8. Solicitudes de cambio de horario 11

3.9. Consultas y reportes 11

4. Requerimientos No Funcionales **13**

5. Modelo Entidad-Relación Extendido (MERX) **14**

6. Normalización de la base de datos **17**

6.1. Esquema universal 17

6.2. Conjunto completo de dependencias funcionales 17

6.3. Cubrimiento mínimo 19

6.4. Relaciones obtenidas por síntesis 19

6.5. Relaciones incorporadas desde el MERX 21

6.6. Clave candidata del esquema universal 22

6.7. Tercera forma normal . . . 23
6.8. Preservación de dependencias funcionales . . . 23
6.9. Pérdida nula de unión . . . 24
6.9.1. Descomposición formal . . . 24
6.9.2. Descomposición física . . . 24
6.10. Conclusiones . . . 24

7. Restricciones de integridad **25**
7.1. Dominios enumerados . . . 25
7.2. Restricciones por relación . . . 25
7.2.1. PERSONA . . . 25
7.2.2. ESTUDIANTE . . . 25
7.2.3. COORDINADOR . . . 26
7.2.4. JEFE_DEPTO . . . 26
7.2.5. PROGRAMA_ACADEMICO . . . 26
7.2.6. ASIGNATURA . . . 26
7.2.7. PROGRAMA_ASIGNATURA . . . 26
7.2.8. MATRICULA . . . 26
7.2.9. BLOQUE_CONTENIDO . . . 27
7.2.10. BLOQUE_PROGRAMA . . . 27
7.2.11. COORDINADOR_EN_PROGRAMA . . . 27
7.2.12. FECHA . . . 27
7.2.13. AULA . . . 27
7.2.14. HORARIO . . . 28
7.2.15. BLOQUE_EN_HORARIO . . . 28
7.2.16. HORARIO_ESTUDIANTE . . . 28
7.2.17. ASISTENCIA . . . 29
7.2.18. VALIDACION . . . 29
7.2.19. REVISION . . . 29
7.2.20. SOLICITUD_CAMBIO . . . 30
7.2.21. ESQUEMA_LLAVE . . . 30
7.3. Restricciones de integridad entre relaciones . . . 30
7.4. Integración con el almacén analítico . . . 31

2. Requerimientos Funcionales

2.1. Gestión de personas y roles

RF01.
Registrar y actualizar los datos de las personas que participan en el sistema, incluyendo estudiantes, coordinadores académicos y jefes de departamento.

RF02.
Registrar y actualizar la edad y el grupo de los estudiantes, así como la especialidad de los coordinadores académicos.

RF03.
Asignar un coordinador académico a cada estudiante y permitir consultar dicha asignación.

RF04.
Gestionar las autorizaciones de los coordinadores académicos por programa académico, indicando el período de vigencia y si están autorizados para generar horarios, validarlos o realizar ambas operaciones.

2.2. Gestión de programas académicos y asignaturas

RF05.
Registrar y actualizar los programas académicos y los planes de estudio correspondientes.

RF06.
Registrar y actualizar las asignaturas y asociarlas a los programas académicos correspondientes.

RF07.
Registrar y consultar la matrícula de los estudiantes en los programas académicos.

2.3. Gestión de bloques de contenido

RF08.
Registrar bloques de contenido, almacenando su descripción y el coordinador académico que los creó.

RF09.
Permitir que cada coordinador académico clasifique los bloques de contenido creados por él según la asignatura, el tipo de actividad (teórica, práctica o laboratorio) y el nivel de complejidad (bajo, medio o alto).

RF10.
Asociar los bloques de contenido con los programas académicos en los que serán utilizados.

RF11.
Impedir la existencia de bloques de contenido duplicados de acuerdo con el criterio de duplicidad establecido.

2.4. Generación y programación de horarios

RF12.
Permitir seleccionar los bloques de contenido y establecer los criterios para la generación del horario, incluyendo el nivel de complejidad, la proporción de bloques por tipo de actividad, la cobertura de las asignaturas y la cantidad total de bloques.

RF13.
Generar automáticamente horarios para un programa académico a partir de los bloques y criterios seleccionados, verificando que el coordinador académico posea la autorización correspondiente.

RF14.
Almacenar, junto al horario generado, el coordinador académico que lo creó, el programa académico, la fecha de creación, el período académico, el estado y los parámetros utilizados durante la generación.

RF15.
Permitir que un coordinador académico utilice bloques de contenido creados por otros coordinadores cuando estos correspondan al programa académico seleccionado.

RF16.
Programar la impartición de cada bloque dentro de un horario, indicando la fecha, la hora de inicio, la hora de fin y el aula correspondiente.

RF17.
Detectar e impedir solapamientos de imparticiones en una misma aula.

2.5. Revisión y validación de horarios

RF18.
Permitir al jefe de departamento revisar un horario generado y registrar la fecha de revisión, las observaciones y la decisión tomada.

RF19.
Permitir al jefe de departamento aprobar un horario o solicitar la generación de otro, registrando los criterios establecidos para la nueva generación.

RF20.
Asociar cada horario generado como resultado de una revisión con la revisión que dio origen a dicha generación.

RF21.
Permitir que los coordinadores académicos validen los horarios de los programas para los cuales están autorizados, almacenando la autorización que realizó la validación, la fecha y las observaciones correspondientes.

RF22.
Actualizar el estado del horario de acuerdo con los procesos de revisión y validación, conservando los registros correspondientes.

2.6. Asignación y consulta de horarios

RF23.
Asignar a los estudiantes los horarios aprobados correspondientes a los programas académicos en los que están matriculados.

RF24.
Impedir que un estudiante tenga asignaciones vigentes cuyos períodos se solapen para horarios distintos del mismo programa y período académico.

RF25.
Permitir a los estudiantes consultar los horarios que les han sido asignados, organizados por programa académico.

RF26.
Permitir a los coordinadores académicos consultar los horarios de los programas que atienden, incluyendo los horarios generados por otros coordinadores del mismo programa.

RF27.
Permitir a estudiantes y coordinadores académicos consultar la disponibilidad y ocupación de los horarios y las aulas.

2.7. Gestión de asistencia

RF28.
Registrar la asistencia de cada estudiante a cada impartición, identificando el horario, el bloque y la fecha de impartición, así como el estado y la fecha del registro de asistencia.

RF29.
Permitir consultar la asistencia por estudiante, horario y bloque, respetando los permisos de acceso correspondientes.

2.8. Gestión de solicitudes de cambio de horario

RF30.
Permitir a los estudiantes realizar solicitudes virtuales de cambio de horario, almacenando la fecha de la solicitud, el motivo y el horario original.

RF31.
Permitir al estudiante solicitar el apoyo de un coordinador académico adicional al que tiene asignado, identificando al coordinador solicitado.

RF32.
Permitir a los coordinadores académicos gestionar y resolver las solicitudes de cambio de horario, almacenando el coordinador que las resuelve, el estado, la fecha de resolución y el resultado.

RF33.
Almacenar el horario resultante de una solicitud de cambio aprobada y actualizar las asignaciones correspondientes, verificando que no se produzcan conflictos y conservando la relación con el horario original y la solicitud.

RF34.
Permitir a los estudiantes consultar el estado y el resultado de sus solicitudes de cambio

de horario.

2.9. Consultas y reportes

RF35.
Obtener el listado de horarios generados automáticamente para un programa académico específico, indicando el nombre del creador, la fecha de creación y los parámetros utilizados.

RF36.
Obtener los bloques de contenido más impartidos en los horarios finales de un programa académico, clasificados por nivel de complejidad y asignatura.

RF37.
Listar los horarios que fueron validados por un coordinador determinado, indicando la fecha de validación y las observaciones realizadas durante el proceso.

RF38.
Generar un reporte sobre el desempeño de los estudiantes en un horario, clasificando los bloques por nivel de complejidad y comparando las tasas de asistencia.

RF39.
Comparar los horarios generados para diferentes programas académicos, verificando la distribución de bloques por asignatura y nivel de complejidad, así como el cumplimiento de los criterios de equilibrio.

RF40.
Determinar, para cada programa académico, la correlación entre el nivel de complejidad de los bloques y el rendimiento promedio de los estudiantes, utilizando la asistencia como indicador de rendimiento.

RF41.
Identificar los 10 bloques de contenido con mayor tasa de inasistencia, indicando el programa académico y el coordinador que los creó.

RF42.
Comparar el desempeño de los estudiantes que solicitaron cambios de horario con el promedio general de asistencia de sus respectivos grupos, para los mismos programas académicos y períodos.

RF43.
Presentar los resultados de las consultas y reportes mediante tablas y gráficos.

RF44.
Permitir ordenar cada columna de las tablas de resultados de acuerdo con los intereses del usuario.

RF45.
Permitir a todos los tipos de usuario exportar a formato PDF la información que tienen autorización para consultar.

2.10. Integración con el almacén de datos histórico

RF46.
Transferir y sincronizar periódicamente los registros de asistencia y ocupación de aulas desde la base de datos transaccional hacia el almacén de datos histórico para su análisis agregado multidimensional.

3. Requerimientos Informacionales

3.1. Gestión de personas y roles

RI01.
Almacenar el identificador único y el nombre de cada persona que participa en el sistema, diferenciando su rol como estudiante, coordinador académico o jefe de departamento.

RI02.
Almacenar la edad y el grupo de cada estudiante, así como el coordinador académico que tiene asignado.

RI03.
Almacenar el identificador, nombre y especialidad de cada coordinador académico.

RI04.
Almacenar, para cada autorización de un coordinador académico sobre un programa, el período de vigencia y si está autorizado para generar horarios, validarlos o realizar ambas operaciones.

3.2. Programas académicos y asignaturas

RI05.
Almacenar el identificador único, nombre y plan de estudios correspondiente a cada programa académico.

RI06.
Almacenar las asignaturas asociadas a cada programa académico y la información propia de cada asignatura.

RI07.
Almacenar la matrícula de cada estudiante en los programas académicos correspondientes.

3.3. Bloques de contenido

RI09.
Almacenar para cada bloque de contenido su identificador, descripción, tipo de actividad y nivel de complejidad.

RI10.
Asociar cada bloque de contenido con la asignatura a la que corresponde, el coordinador que lo creó y los programas académicos en los que puede ser utilizado.

RI11.
Mantener la información necesaria para identificar y evitar bloques de contenido duplicados conforme al criterio de duplicidad establecido.

3.4. Horarios y programación

RI12.
Almacenar para cada horario su identificador, fecha de creación, estado, período académico, programa académico al que pertenece y coordinador que lo generó.

RI13.
Almacenar los parámetros utilizados para la generación de cada horario, incluyendo el nivel de complejidad, la proporción de bloques teóricos, prácticos y de laboratorio, la cobertura de las asignaturas y la cantidad total de bloques.

RI14.
Almacenar cada impartición programada de un bloque mediante su asociación con un horario y una fecha, indicando la hora de inicio, la hora de fin y el aula correspondiente.

RI15.
Permitir identificar la composición de cada horario, incluyendo los bloques de contenido, sus asignaturas, tipos de actividad y niveles de complejidad, así como las fechas, horas y aulas en que son impartidos.

RI16.
Permitir consultar la ocupación de las aulas a partir de las imparticiones programadas, considerando el edificio, la fecha y el intervalo horario correspondiente.

3.5. Revisión y validación

RI17.
Almacenar para cada revisión de un horario el jefe de departamento que la realiza, la fecha, las observaciones y la decisión tomada.

RI18.
Almacenar los criterios establecidos por el jefe de departamento cuando solicite la generación de un nuevo horario y mantener la relación con el horario generado como resultado de dicha revisión.

RI19.
Almacenar para cada validación la autorización del coordinador que la realiza, el horario validado, la fecha y las observaciones correspondientes.

RI20.
Mantener la información necesaria para distinguir los horarios según su estado de revisión y validación.

3.6. Asignación y consulta de horarios

RI21.
Mantener el historial de asignaciones de horarios a estudiantes, identificando cada asignación por el estudiante, el horario y la fecha de asignación, y almacenando las fechas de inicio y finalización de su vigencia.

RI22.
Permitir consultar los horarios asignados a cada estudiante organizados por programa académico.

RI23.
Permitir a los coordinadores consultar los horarios correspondientes a los programas que atienden, incluyendo la información de su creador, fecha de creación, período y estado.

RI24.
Proporcionar la información de fechas e intervalos horarios de las sesiones asignadas a

cada estudiante para identificar asignaciones repetidas.

3.7. Asistencia

RI25.
Almacenar para cada registro de asistencia el estudiante, la impartición correspondiente, el estado de asistencia y la fecha en que se realizó el registro.

RI26.
Permitir consultar los registros de asistencia de un estudiante según el horario y el bloque de contenido correspondiente.

RI27.
Diferenciar la ausencia de un registro de asistencia de una ausencia confirmada, de manera que la falta de un registro no se interprete automáticamente como una inasistencia.

3.8. Solicitudes de cambio de horario

RI28.
Almacenar para cada solicitud de cambio de horario el estudiante que la realiza, la fecha, el motivo y el horario original.

RI29.
Identificar el coordinador académico adicional al que el estudiante solicita apoyo para atender la solicitud.

RI30.
Almacenar el coordinador académico que resuelve la solicitud, su estado, la fecha de resolución y el resultado obtenido.

RI31.
Mantener la relación entre la solicitud de cambio, el horario original y el horario resultante.

3.9. Consultas y reportes

RI32.
Proporcionar la información necesaria para listar los horarios generados automáticamente para un programa académico, incluyendo el creador, la fecha de creación y los parámetros utilizados.

RI33.
Proporcionar la información necesaria para determinar los bloques de contenido más impartidos en los horarios finales de un programa académico, clasificándolos por nivel de complejidad y asignatura. El conteo corresponderá a las imparticiones del bloque.

RI34.
Proporcionar la información necesaria para listar los horarios validados por un coordinador determinado, incluyendo la fecha de validación y las observaciones realizadas.

RI35.
Proporcionar la información necesaria para generar reportes sobre el desempeño de los estudiantes en un horario, clasificando los bloques por nivel de complejidad y comparando las tasas de asistencia.

RI36.
Proporcionar la información necesaria para comparar horarios generados para diferentes programas académicos, considerando la distribución de bloques por asignatura y nivel de complejidad y el cumplimiento de los criterios de equilibrio.

RI37.
Mostrar, para cada programa académico y período consultado, el nivel de complejidad de los bloques, el rendimiento promedio medido mediante asistencia y la correlación entre ambos indicadores.

RI38.
Identificar los 10 bloques de contenido con mayor tasa de inasistencia, indicando el coordinador que los creó y el programa académico al que pertenecen.

RI39.
Comparar el desempeño de los estudiantes que solicitaron cambios de horario con el promedio general de asistencia de sus respectivos grupos, considerando los mismos programas académicos y períodos.

RI40.
Proporcionar los resultados de las consultas y reportes en forma de tablas y gráficos, así como la información necesaria para ordenar los resultados y exportar a formato PDF aquellos datos que el usuario esté autorizado a consultar.

RI41.
Almacenar en una estructura de datos independiente y optimizada para lectura masiva (ClickHouse) la información histórica de ocupación de aulas y asistencia acumulada por bloque, organizada por franja horaria, edificio y período académico.

4. Requerimientos No Funcionales

RNF01. Rendimiento en consultas de disponibilidad.
El sistema debe responder en menos de un segundo a las consultas de disponibilidad de horarios y aulas realizadas concurrentemente por estudiantes y coordinadores durante los períodos de matrícula.
Solución técnica: implementar mecanismos de caché en memoria en la capa de aplicación/API (.NET Core) para las consultas de alta demanda y crear índices optimizados en PostgreSQL sobre la disponibilidad de aulas e imparticiones.

RNF02. Almacenamiento y procesamiento analítico.
El almacenamiento de datos analíticos históricos de gran volumen debe estar desacoplado y organizado en una estructura orientada a columnas (ClickHouse). Esto debe permitir ejecutar consultas agregadas masivas —como promedios, sumas y correlaciones— con bajo consumo de recursos en el motor relacional primario (PostgreSQL).

RNF03. Arquitectura y facilidad de prueba.
El sistema debe diseñarse siguiendo el patrón de Arquitectura Limpia (Clean Architecture), aislando el dominio y las reglas de negocio de los detalles de infraestructura (PostgreSQL/EF Core para OLTP y ClickHouse para OLAP), y asegurando la extensibilidad y facilidad para realizar pruebas unitarias del back-end y del front-end.

5. Modelo Entidad-Relación Extendido (MERX)

Las siguientes figuras presentan el MERX completo en el orden correspondiente.

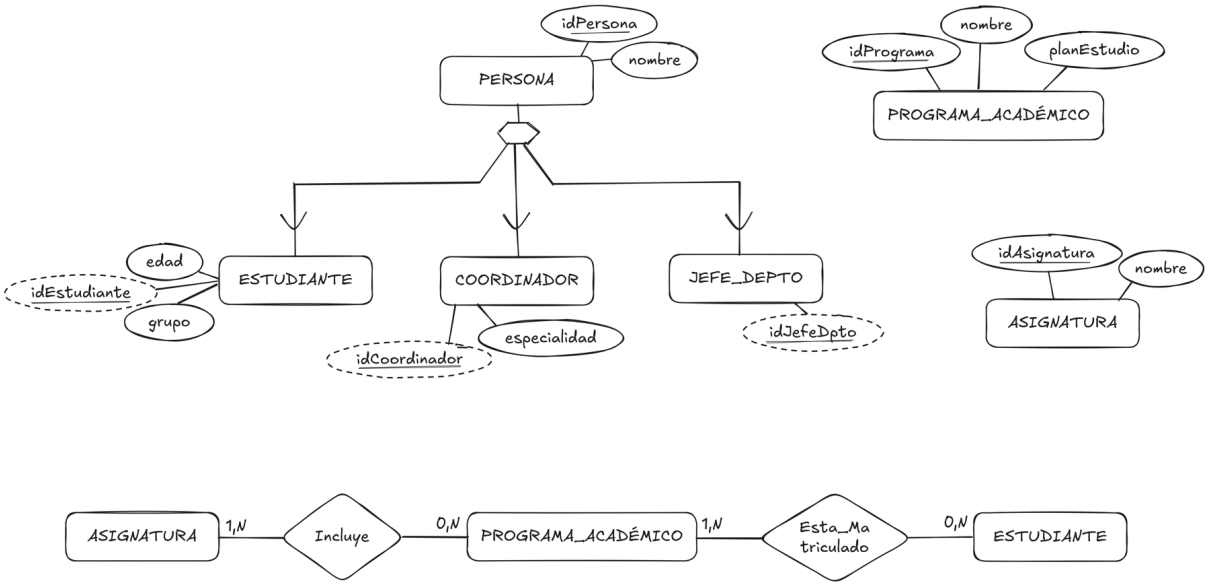


Figura 1: MERX – Parte 1. Entidades principales y relaciones iniciales.

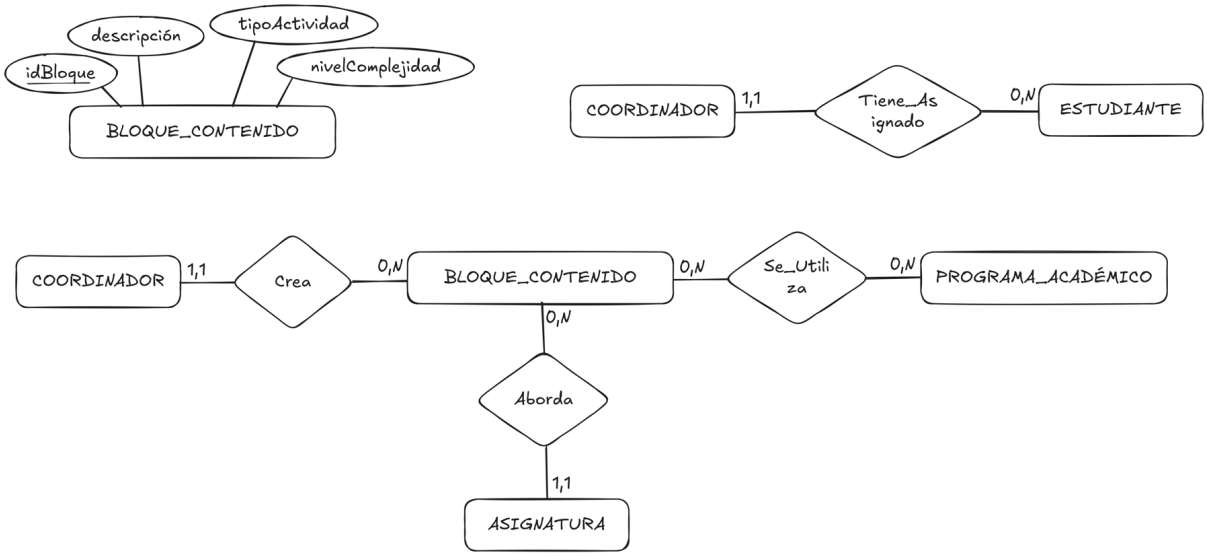


Figura 2: MERX – Parte 2. Bloques de contenido y asociaciones.

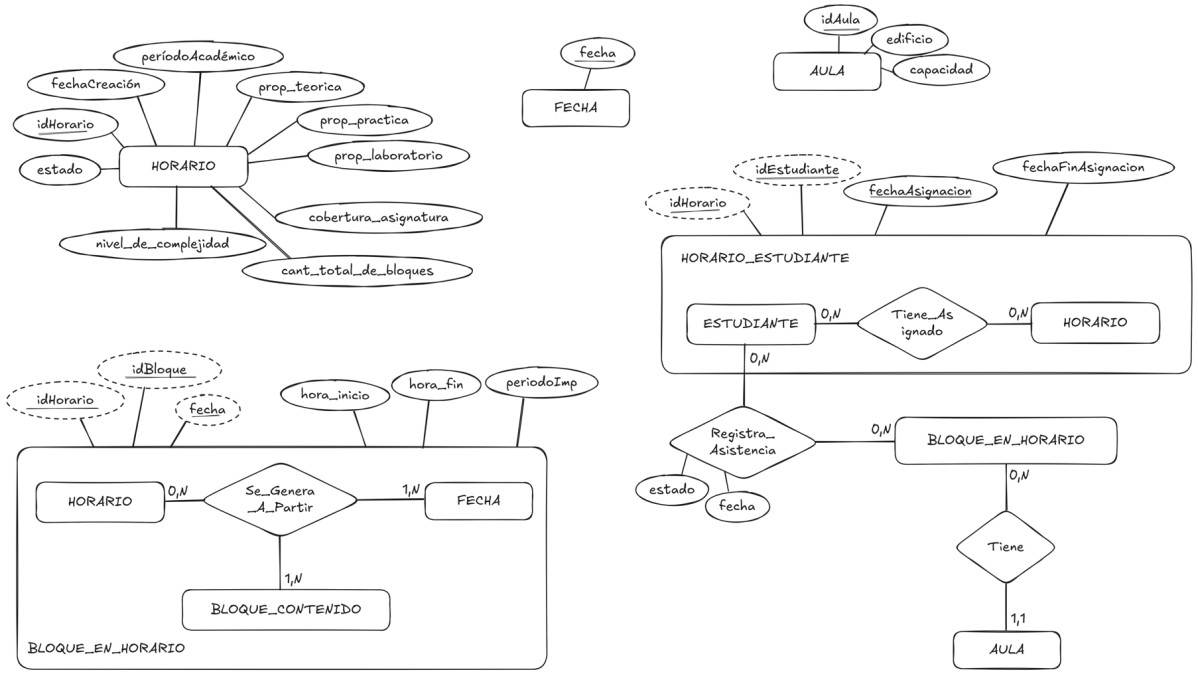


Figura 3: MERX – Parte 3. Horarios, fechas, aulas, asignaciones y asistencia.

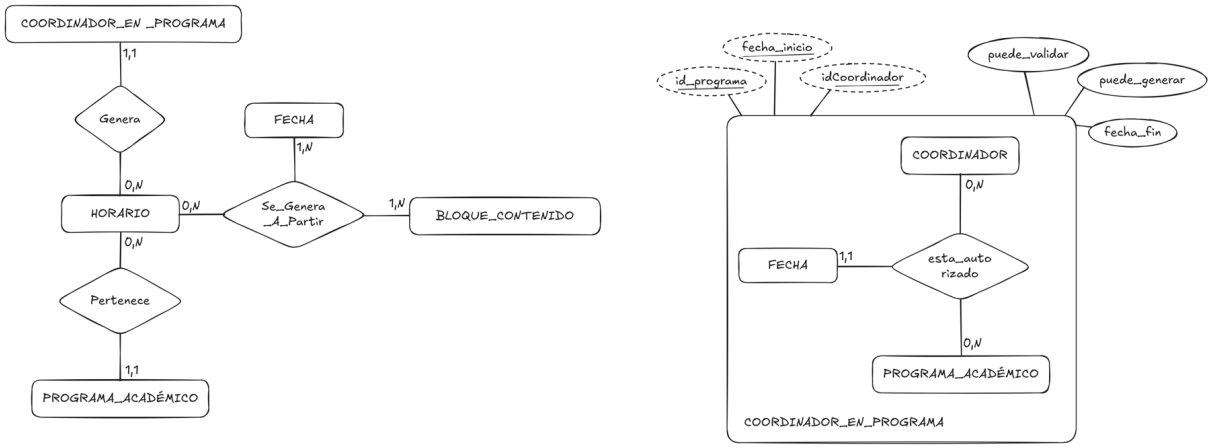


Figura 4: MERX – Parte 4. Generación de horarios y autorización de coordinadores.

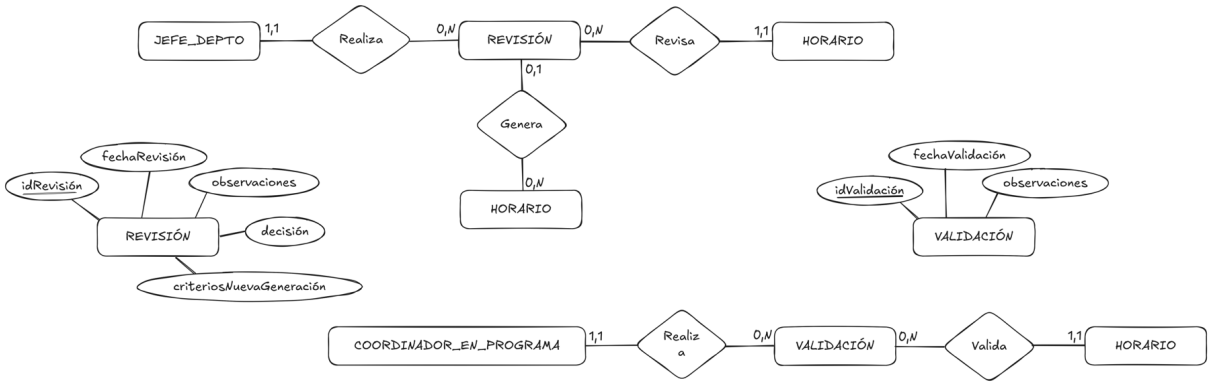


Figura 5: MERX – Parte 5. Revisión y validación de horarios.

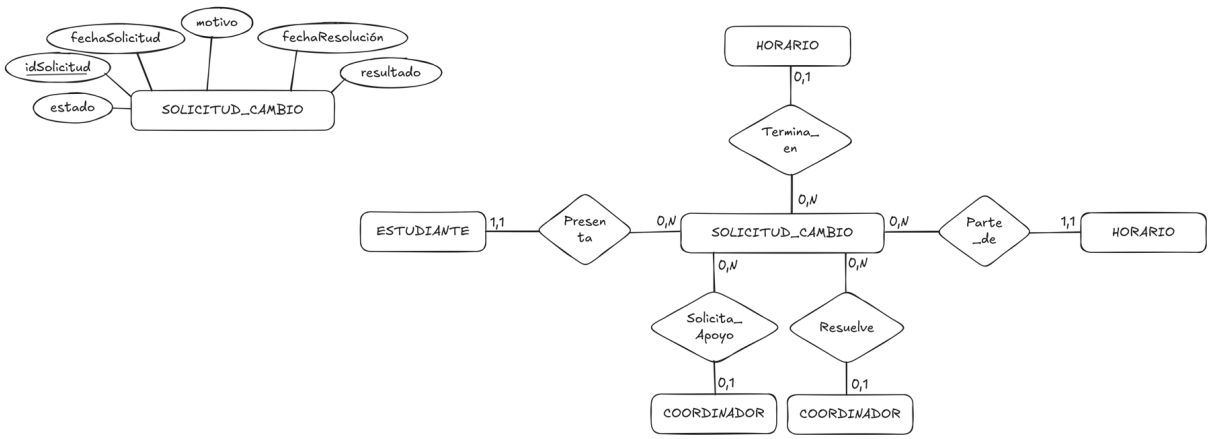


Figura 6: MERX – Parte 6. Solicitudes de cambio de horario.

6. Normalización de la base de datos

6.1. Esquema universal

El esquema universal utilizado en el análisis es:

$$
\begin{aligned}
U = \{ & idPersona, nombrePersona, idEstudiante, edad, \\
& grupo, idCoordinadorAsig, idCoordinador, especialidad, \\
& idJefeDepto, idPrograma, nombrePrograma, planEstudio, \\
& idAsignatura, nombreAsignatura, idBloque, descripcion, \\
& tipoActividad, nivelComplejidad, idCoordinadorCreador, \\
& fechaInicio, fechaFin, puedeGenerar, puedeValidar, \\
& fecha, idAula, edificio, capacidad, idHorario, \\
& fechaCreacion, periodoAcademico, estadoHorario, \\
& nivelComplejidadHorario, cantTotalBloques, propTeorica, \\
& propPractica, propLaboratorio, coberturaAsignatura, \\
& idCoordinadorGen, fechaInicioGen, idRevisionOrigen, \\
& horaInicio, horaFin, periodoImp, fechaAsignacion, \\
& fechaFinAsignacion, estadoAsistencia, fechaRegistro, \\
& idValidacion, fechaValidacion, observacionesValidacion, \\
& idCoordinadorVal, fechaInicioVal, idRevision, fechaRevision, \\
& observacionesRevision, decision, criteriosNuevaGeneracion, \\
& idSolicitud, fechaSolicitud, estadoSolicitud, motivo, \\
& fechaResolucion, resultado, idHorarioOriginal, idHorarioNuevo, \\
& idCoordinadorApoyo, idCoordinadorResolutor\}.
\end{aligned}
$$

Las claves foráneas y las restricciones de inclusión se consideran restricciones de integridad adicionales; no forman parte de F, salvo que expresen una dependencia funcional entre atributos de una misma relación.

6.2. Conjunto completo de dependencias funcionales

Para el análisis se considera el conjunto completo de dependencias funcionales aprobado para el modelo:

$$F = \bigcup_{i=1}^{17} F_i.$$

F_1 : $idPersona \to nombrePersona$

F_2 : $idEstudiante \to edad, grupo, idCoordinadorAsig$

F_3 : $idCoordinador \to especialidad$

F_4 : $idPrograma \to nombrePrograma, planEstudio$

F_5 : $idAsignatura \to nombreAsignatura$

$$
\begin{aligned}
F_6: \quad & idBloque \rightarrow descripcion, tipoActividad, nivelComplejidad, \\
& \qquad idAsignatura, idCoordinadorCreador \\
F_7: \quad & (idAsignatura, tipoActividad, descripcion) \rightarrow idBloque \\
F_8: \quad & (idCoordinador, idPrograma, fechaInicio) \rightarrow fechaFin, puedeGenerar, \\
& \qquad puedeValidar \\
F_9: \quad & idAula \rightarrow edificio, capacidad \\
F_{10}: \quad & idHorario \rightarrow fechaCreacion, periodoAcademico, estadoHorario, \\
& \qquad nivelComplejidadHorario, cantTotalBloques, \\
& \qquad propTeorica, propPractica, propLaboratorio, \\
& \qquad coberturaAsignatura, idPrograma, idCoordinadorGen, \\
& \qquad fechaInicioGen, idRevisionOrigen \\
F_{11}: \quad & (idHorario, idBloque, fecha) \rightarrow horaInicio, horaFin, periodoImp, idAula \\
F_{12}: \quad & (idAula, fecha, horaInicio) \rightarrow idHorario, idBloque \\
F_{13}: \quad & (idEstudiante, idHorario, fechaAsignacion) \rightarrow fechaFinAsignacion \\
F_{14}: \quad & (idEstudiante, idHorario, idBloque, fecha) \rightarrow estadoAsistencia, fechaRegistro \\
F_{15}: \quad & idValidacion \rightarrow fechaValidacion, observacionesValidacion, \\
& \qquad idHorario, idCoordinadorVal, fechaInicioVal \\
F_{16}: \quad & idRevision \rightarrow fechaRevision, observacionesRevision, decision, \\
& \qquad criteriosNuevaGeneracion, idJefeDepto, idHorario \\
F_{17}: \quad & idSolicitud \rightarrow fechaSolicitud, estadoSolicitud, motivo, fechaResolucion, \\
& \qquad resultado, idEstudiante, idHorarioOriginal, idHorarioNuevo, \\
& \qquad idCoordinadorApoyo, idCoordinadorResolutor
\end{aligned}
$$

Los atributos de rol se distinguen explícitamente. Por ejemplo, $idValidacion$ determina $idCoordinadorVal$ e $idHorario$, mientras que $idHorario$ determina $idCoordinadorGen$. No se identifica al validador con el generador. Las dependencias derivables por transitividad se entienden como consecuencias de F^+ y no se repiten como dependencias independientes del cubrimiento.

Las dependencias F_7, F_8, F_{11}, F_{12}, F_{13} y F_{14} expresan hipótesis de negocio cuya validez se asume para este análisis:

1. F_7 presupone que $(idAsignatura, tipoActividad, descripcion)$ identifica un único bloque.
2. F_8 presupone que $(idCoordinador, idPrograma, fechaInicio)$ identifica una autorización concreta.
3. F_{11} presupone que $(idHorario, idBloque, fecha)$ identifica una única impartición.
4. F_{12} presupone que una misma aula no puede tener dos imparticiones con la misma fecha y hora de inicio. Esta dependencia no expresa por sí sola la prohibición general de solapamiento entre intervalos de distinta duración.
5. F_{13} presupone que $(idEstudiante, idHorario, fechaAsignacion)$ identifica un registro de asignación. La inclusión de la fecha permite conservar asignaciones históricas y registrar una nueva asignación del mismo horario tras cerrar la anterior.

6. F_{14} presupone que la asistencia se identifica mediante $idEstudiante$, $idHorario$, $idBloque$ y $fecha$.

6.3. Cubrimiento mínimo

Se descomponen los consecuentes de F para obtener dependencias atómicas, cada una con un solo atributo a la derecha. El total es 62:

Grupo	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9
Cantidad	1	3	1	2	1	5	1	3	2

Grupo	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}	F_{16}	F_{17}
Cantidad	13	4	2	1	2	5	6	10

Para comprobar que no hay atributos extraños en los determinantes, se elimina por turno cada atributo de cada determinante compuesto y se calcula el cierre del determinante restante respecto de F. Los determinantes compuestos son los de F_7, F_8, F_{11}, F_{12}, F_{13} y F_{14}. En cada prueba, el atributo derecho de la dependencia correspondiente no pertenece al cierre obtenido; por tanto, ninguno de esos atributos es extraño.

Para comprobar que no hay dependencias redundantes, se retira por turno cada dependencia atómica $X \to A$ y se calcula X^+ respecto del conjunto restante. La dependencia es necesaria cuando $A \notin X^+$. Este procedimiento se aplica a las 62 dependencias atómicas.

Se comprueban especialmente los consecuentes que aparecen en más de una dependencia:

- Para $idBloque$, se verifica que al retirar las dependencias con ese consecuente, sus determinantes no pueden obtener $idBloque$ usando la otra dependencia que también lo determina.
- Para $idHorario$, se verifica por separado la necesidad de las dependencias atómicas procedentes de F_{12}, F_{15} y F_{16}.

El cálculo de cierres no identifica dependencias redundantes. Por tanto, bajo las hipótesis declaradas, el conjunto de 62 dependencias atómicas constituye un cubrimiento mínimo de F.

6.4. Relaciones obtenidas por síntesis

La síntesis agrupa las dependencias con el mismo determinante. Las relaciones obtenidas y sus claves candidatas son:

R_1 PERSONA

$$R_1 = (idPersona, nombrePersona)$$

Clave candidata: $idPersona$.

R_2 **ESTUDIANTE**

$$R_2 = (idEstudiante, edad, grupo, idCoordinadorAsig)$$

Clave candidata: $idEstudiante$.

R_3 **COORDINADOR**

$$R_3 = (idCoordinador, especialidad)$$

Clave candidata: $idCoordinador$.

R_4 **PROGRAMA_ACADEMICO**

$$R_4 = (idPrograma, nombrePrograma, planEstudio)$$

Clave candidata: $idPrograma$.

R_5 **ASIGNATURA**

$$R_5 = (idAsignatura, nombreAsignatura)$$

Clave candidata: $idAsignatura$.

R_6 **BLOQUE_CONTENIDO**

$$\begin{aligned} R_6 = (&idBloque, descripcion, tipoActividad,\\ &nivelComplejidad, idAsignatura,\\ &idCoordinadorCreador) \end{aligned}$$

Claves candidatas: $idBloque$ y $(idAsignatura, tipoActividad, descripcion)$.

R_7 **COORDINADOR_EN_PROGRAMA**

$$\begin{aligned} R_7 = (&idCoordinador, idPrograma, fechaInicio,\\ &fechaFin, puedeGenerar, puedeValidar) \end{aligned}$$

Clave candidata: $(idCoordinador, idPrograma, fechaInicio)$.

R_8 **AULA**

$$R_8 = (idAula, edificio, capacidad)$$

Clave candidata: $idAula$.

R_9 **HORARIO**

$$\begin{aligned} R_9 = (&idHorario, fechaCreacion, periodoAcademico,\\ &estadoHorario, nivelComplejidadHorario,\\ &cantTotalBloques, propTeorica, propPractica,\\ &propLaboratorio, coberturaAsignatura,\\ &idPrograma, idCoordinadorGen,\\ &fechaInicioGen, idRevisionOrigen) \end{aligned}$$

Clave candidata: $idHorario$.

R_{10} **BLOQUE_EN_HORARIO**

$$R_{10} = (idHorario, idBloque, fecha, horaInicio, horaFin, periodoImp, idAula)$$

Claves candidatas: $(idHorario, idBloque, fecha)$ y $(idAula, fecha, horaInicio)$.

R_{11} **HORARIO_ESTUDIANTE**

$$R_{11} = (idEstudiante, idHorario, fechaAsignacion, fechaFinAsignacion)$$

Clave candidata: $(idEstudiante, idHorario, fechaAsignacion)$.

R_{12} **ASISTENCIA**

$$R_{12} = (idEstudiante, idHorario, idBloque, fecha, estadoAsistencia, fechaRegistro)$$

Clave candidata: $(idEstudiante, idHorario, idBloque, fecha)$.

R_{13} **VALIDACION**

$$R_{13} = (idValidacion, fechaValidacion, observacionesValidacion, idHorario, idCoordinadorVal, fechaInicioVal)$$

Clave candidata: $idValidacion$.

R_{14} **REVISION**

$$R_{14} = (idRevision, fechaRevision, observacionesRevision, decision, criteriosNuevaGeneracion, idJefeDepto, idHorario)$$

Clave candidata: $idRevision$.

R_{15} **SOLICITUD_CAMBIO**

$$R_{15} = (idSolicitud, fechaSolicitud, estadoSolicitud, motivo, fechaResolucion, resultado, idEstudiante, idHorarioOriginal, idHorarioNuevo, idCoordinadorApoyo, idCoordinadorResolutor)$$

Clave candidata: $idSolicitud$.

6.5. Relaciones incorporadas desde el MERX

La transformación del MERX incorpora además las siguientes relaciones, que no son generadas por dependencias funcionales no triviales de F:

R_{17} **PROGRAMA_ASIGNATURA**

$$R_{17} = (idPrograma, idAsignatura)$$

Clave compuesta.

R_{18} **MATRICULA**

$$R_{18} = (idEstudiante, idPrograma)$$

Clave compuesta.

R_{19} **BLOQUE_PROGRAMA**

$$R_{19} = (idBloque, idPrograma)$$

Clave compuesta.

R_{20} **FECHA**

$$R_{20} = (fecha)$$

Clave candidata: $fecha$.

R_{21} **JEFE_DEPTO**

$$R_{21} = (idJefeDepto)$$

Clave candidata: $idJefeDepto$; clave foránea a PERSONA.

Estas relaciones se incorporan al esquema final por la transformación de asociaciones, entidades y subtipos del MERX. En las asociaciones binarias R_{17}–R_{19}, la clave comprende ambos identificadores.

6.6. Clave candidata del esquema universal

Los atributos que no aparecen en el consecuente de ninguna dependencia de F son:

$$N = \{idPersona, idCoordinador, fechaInicio, fecha, \\ fechaAsignacion, idValidacion, idRevision, idSolicitud\}.$$

Todos deben formar parte de cualquier clave candidata respecto de F. También se necesita $idBloque$: al calcular el cierre de los otros atributos propuestos para K_U, no se obtiene $idBloque$.

Se propone:

$$K_U = \{idPersona, idCoordinador, fechaInicio, fecha, \\ fechaAsignacion, idBloque, idValidacion, \\ idRevision, idSolicitud\}.$$

El cierre de K_U contiene todos los atributos de U. $idPersona$ determina $nombrePersona$. $idValidacion$ determina los datos de validación e $idHorario$; este último determina los datos del horario, incluidos $idPrograma$ e $idCoordinadorGen$. $idRevision$ determina los datos de revisión e $idJefeDepto$. $idSolicitud$ determina sus datos, incluidos $idEstudiante$,

los horarios implicados y los coordinadores asociados. $idEstudiante$ determina sus propios atributos, e $idCoordinador$ determina $especialidad$.

Con $idCoordinador, idPrograma, fechaInicio$ se obtienen los atributos de la autorización. $idBloque$ determina los datos del bloque e $idAsignatura$, que determina $nombreAsignatura$. Finalmente, $(idHorario, idBloque, fecha)$ determina los datos de la impartición, e $idAula$ determina $edificio$ y $capacidad$. Al estar disponibles $idEstudiante$, $idHorario$ y $fechaAsignacion$, F_{13} determina $fechaFinAsignacion$. También quedan disponibles los determinantes de asistencia. Por tanto:

$$K_U^+ = U.$$

La clave es mínima: cada atributo de N es obligatorio por no aparecer en ningún consecuente, y $idBloque$ no se obtiene del cierre de los restantes atributos de K_U. Por tanto, K_U es una clave candidata de U respecto de F.

Para aplicar el criterio clásico de síntesis sin pérdida se incorpora:

$$R_{16} = \text{ESQUEMA_LLAVE}(K_U).$$

En este diseño, R_{16} se materializa como una relación física y contiene los atributos de K_U, cuya combinación forma su clave candidata.

6.7. Tercera forma normal

El teorema de síntesis en 3FN garantiza que, a partir de un cubrimiento mínimo, las relaciones generadas por la síntesis están en 3FN. Las relaciones R_1–R_{15} se obtienen agrupando dependencias con el mismo determinante. En R_6 y R_{10} se conservan las claves candidatas alternativas derivadas de las dependencias funcionales correspondientes. En R_{11} la clave candidata incluye la fecha de asignación, como especifica el nuevo F_{13}.

Las relaciones incorporadas desde el MERX se verifican por separado. En R_{17}, las clausuras de $idPrograma$ e $idAsignatura$, proyectadas sobre la relación, no contienen el otro atributo. En R_{18} ocurre lo mismo para $idEstudiante$ e $idPrograma$, y en R_{19} para $idBloque$ e $idPrograma$. Por tanto, sus claves candidatas son los pares completos y no tienen dependencias funcionales no triviales en sus proyecciones. Las relaciones R_{20} y R_{21} contienen un único atributo, por lo que no pueden tener dependencias funcionales no triviales. La relación R_{16} contiene únicamente los atributos de K_U, que constituyen su clave candidata; por tanto, satisface BCNF y, en consecuencia, 3FN.

Por el teorema de síntesis y la revisión de las relaciones adicionales, el esquema satisface 3FN respecto de F. Esta conclusión no cubre las restricciones de inclusión, temporales ni las reglas de negocio entre relaciones.

6.8. Preservación de dependencias funcionales

Sea la descomposición física implementada

$$D_{\text{fisica}} = \{R_1, \ldots, R_{21}\} \qquad \text{y} \qquad F' = \bigcup_{R_i \in D_{\text{fisica}}} \pi_{R_i}(F).$$

Cada dependencia de F queda contenida en una relación:

Dependencias	Relación	Dependencias	Relación
F_1, F_2	R_1, R_2	F_3	R_3
F_4	R_4	F_5	R_5
F_6, F_7	R_6	F_8	R_7
F_9	R_8	F_{10}	R_9
F_{11}, F_{12}	R_{10}	F_{13}	R_{11}
F_{14}	R_{12}	F_{15}	R_{13}
F_{16}	R_{14}	F_{17}	R_{15}

Por ello, $F \subseteq (F')^+$. Además, cada dependencia de F' es consecuencia de F, por definición de proyección, de modo que $(F')^+ \subseteq F^+$. En consecuencia:

$$(F')^+ = F^+,$$

y la descomposición preserva las dependencias funcionales de F.

6.9. Pérdida nula de unión

6.9.1. Descomposición formal

La síntesis clásica en 3FN aplicada a un cubrimiento mínimo produce una descomposición sin pérdida si se incorpora una relación que contenga una clave candidata de U cuando ninguna de las relaciones sintetizadas la contiene. La relación auxiliar R_{16} contiene K_U, por lo que la descomposición formal que incluye R_{16} es sin pérdida respecto de F.

Las relaciones R_{17}–R_{21} se incorporan como proyecciones adicionales de U. Añadirlas a la reunión de una descomposición ya sin pérdida no introduce tuplas espurias.

6.9.2. Descomposición física

La relación R_{16} se materializa en el diseño físico. Por tanto,

$$D_{\text{fisica}} = \{R_1, \ldots, R_{21}\}.$$

Como R_{16} contiene la clave candidata K_U, el criterio de síntesis clásica justifica la pérdida nula de unión de esta descomposición respecto de F.

6.10. Conclusiones

Bajo las hipótesis declaradas, el conjunto F se considera completo para el análisis y sus 62 dependencias atómicas forman un cubrimiento mínimo. La síntesis resultante satisface 3FN y preserva las dependencias funcionales de F. La relación R_{16}, que contiene una clave candidata de U, se materializa en la descomposición física. Por el criterio clásico de síntesis, la descomposición implementada tiene pérdida nula de unión respecto de F.

7. Restricciones de integridad

Las restricciones se expresan como claves, referencias, restricciones de dominio, restricciones de fila o reglas que involucran varias relaciones. Las reglas interrelacionales que no puedan expresarse mediante restricciones declarativas de una sola tabla deben implementarse mediante disparadores o transacciones de aplicación.

7.1. Dominios enumerados

Para evitar dominios indefinidos, se establecen los siguientes valores:

`estadoAsistencia`
`PRESENTE`, `AUSENTE`, `TARDANZA`, `AUSENCIA_JUSTIFICADA`.

`decision` **de REVISION**
`APROBADO`, `SOLICITAR_NUEVA_GENERACION`.

`estadoSolicitud`
`PENDIENTE`, `EN_REVISION`, `APROBADA`, `RECHAZADA`, `CANCELADA`.

`estadoHorario`
`GENERADO`, `EN_REVISION`, `APROBADO`, `REQUIERE_REGENERACION`, `PENDIENTE_VALIDACION`, `VALIDADO`, `RECHAZADO`, `CANCELADO`.

Estos dominios se implementan mediante tipos enumerados o restricciones `CHECK`. La ausencia de un registro de asistencia no equivale a una inasistencia confirmada; solo un registro con estado `AUSENTE` representa una ausencia confirmada.

7.2. Restricciones por relación

7.2.1. PERSONA

- Clave primaria: `idPersona`.
- `nombrePersona` es obligatorio.

7.2.2. ESTUDIANTE

- Clave primaria: `idEstudiante`.
- Clave foránea: `idEstudiante` referencia PERSONA(`idPersona`).
- Clave foránea: `idCoordinadorAsig` referencia COORDINADOR(`idCoordinador`).
- `edad` es positiva.
- `grupo` e `idCoordinadorAsig` son obligatorios.

7.2.3. COORDINADOR

- Clave primaria: `idCoordinador`.
- Clave foránea: `idCoordinador` referencia PERSONA(`idPersona`).
- `especialidad` es obligatoria.

7.2.4. JEFE_DEPTO

- Clave primaria: `idJefeDepto`.
- Clave foránea: `idJefeDepto` referencia PERSONA(`idPersona`).

7.2.5. PROGRAMA_ACADEMICO

- Clave primaria: `idPrograma`.
- `nombrePrograma` y `planEstudio` son obligatorios.

7.2.6. ASIGNATURA

- Clave primaria: `idAsignatura`.
- `nombreAsignatura` es obligatorio.

7.2.7. PROGRAMA_ASIGNATURA

- Clave primaria compuesta: (`idPrograma`, `idAsignatura`).
- Clave foránea: `idPrograma` referencia PROGRAMA_ACADEMICO(`idPrograma`).
- Clave foránea: `idAsignatura` referencia ASIGNATURA(`idAsignatura`).

7.2.8. MATRICULA

- Clave primaria compuesta: (`idEstudiante`, `idPrograma`).
- Clave foránea: `idEstudiante` referencia ESTUDIANTE(`idEstudiante`).
- Clave foránea: `idPrograma` referencia PROGRAMA_ACADEMICO(`idPrograma`).

7.2.9. BLOQUE_CONTENIDO

- Clave primaria: `idBloque`.
- Claves foráneas: `idAsignatura` referencia ASIGNATURA(`idAsignatura`); `idCoordinadorCreado` referencia COORDINADOR(`idCoordinador`).
- Restricción de unicidad: (`idAsignatura`, `tipoActividad`, `descripcion`).
- `tipoActividad` pertenece a {`TEORICA`, `PRACTICA`, `LABORATORIO`}.
- `nivelComplejidad` pertenece a {`BAJO`, `MEDIO`, `ALTO`}.
- `descripcion`, `tipoActividad`, `nivelComplejidad`, `idAsignatura` e `idCoordinadorCreador` son obligatorios.

7.2.10. BLOQUE_PROGRAMA

- Clave primaria compuesta: (`idBloque`, `idPrograma`).
- Claves foráneas: `idBloque` referencia BLOQUE_CONTENIDO(`idBloque`); `idPrograma` referencia PROGRAMA_ACADEMICO(`idPrograma`).

7.2.11. COORDINADOR_EN_PROGRAMA

- Clave primaria compuesta: (`idCoordinador`, `idPrograma`, `fechaInicio`).
- Claves foráneas:
  - `idCoordinador` referencia COORDINADOR(`idCoordinador`).
  - `idPrograma` referencia PROGRAMA_ACADEMICO(`idPrograma`).
  - `fechaInicio` referencia FECHA(`fecha`).
- Si `fechaFin` tiene valor, debe cumplirse `fechaFin` $\geq$ `fechaInicio`.
- `puedeGenerar` y `puedeValidar` son booleanos y no nulos.

7.2.12. FECHA

- Clave primaria: `fecha`.

7.2.13. AULA

- Clave primaria: `idAula`.
- `edificio` es obligatorio.
- `capacidad` es un entero positivo y obligatorio.

7.2.14. HORARIO

- Clave primaria: `idHorario`.
- Clave foránea: `idPrograma` referencia PROGRAMA_ACADEMICO(`idPrograma`).
- Clave foránea compuesta:

  $(idCoordinadorGen, idPrograma, fechaInicioGen)$
  $\rightarrow COORDINADOR_EN_PROGRAMA(idCoordinador, idPrograma, fechaInicio)$.

- Si `idRevisionOrigen` tiene valor, referencia REVISION(`idRevision`).
- `estadoHorario` pertenece al dominio enumerado y es obligatorio.
- `cantTotalBloques` es positivo.
- Las proporciones son valores numéricos entre 0 y 1, inclusive, y cumplen:

$$\texttt{propTeorica} + \texttt{propPractica} + \texttt{propLaboratorio} = 1.$$

- `coberturaAsignatura` es no negativa.

7.2.15. BLOQUE_EN_HORARIO

- Clave primaria: (`idHorario`, `idBloque`, `fecha`).
- Clave candidata alternativa, bajo la hipótesis de F_{12}: (`idAula`, `fecha`, `horaInicio`).
- Claves foráneas:
  - `idHorario` referencia HORARIO(`idHorario`).
  - `idBloque` referencia BLOQUE_CONTENIDO(`idBloque`).
  - `fecha` referencia FECHA(`fecha`).
  - `idAula` referencia AULA(`idAula`).
- `horaInicio`, `horaFin`, `periodoImp` e `idAula` son obligatorios.
- Debe cumplirse `horaInicio` $<$ `horaFin`.
- `periodoImp` pertenece al dominio de períodos de impartición definido por el proyecto.

7.2.16. HORARIO_ESTUDIANTE

- Clave primaria: (`idEstudiante`, `idHorario`, `fechaAsignacion`).
- Claves foráneas:
  - `idEstudiante` referencia ESTUDIANTE(`idEstudiante`).
  - `idHorario` referencia HORARIO(`idHorario`).

- `fechaAsignacion` es obligatoria y tiene precisión suficiente para distinguir registros de asignación diferentes.
- Si `fechaFinAsignacion` tiene valor, debe cumplirse `fechaFinAsignacion` $\geq$ `fechaAsignacion`.

7.2.17. ASISTENCIA

- Clave primaria: (`idEstudiante`, `idHorario`, `idBloque`, `fecha`).
- Clave foránea: `idEstudiante` referencia ESTUDIANTE(`idEstudiante`).
- Clave foránea compuesta:

$$(idHorario, idBloque, fecha) \\ \rightarrow BLOQUE_EN_HORARIO(idHorario, idBloque, fecha).$$

- `estadoAsistencia` pertenece al dominio enumerado.
- `fechaRegistro` y `estadoAsistencia` son obligatorios.

7.2.18. VALIDACION

- Clave primaria: `idValidacion`.
- Clave foránea: `idHorario` referencia HORARIO(`idHorario`).
- `fechaValidacion`, `idCoordinadorVal` y `fechaInicioVal` son obligatorios.
- La vigencia y el permiso de validación del coordinador se comprueban mediante una regla interrelacional que compara la autorización con el programa del horario validado.

7.2.19. REVISION

- Clave primaria: `idRevision`.
- Claves foráneas:
  - `idJefeDepto` referencia JEFE_DEPTO(`idJefeDepto`).
  - `idHorario` referencia HORARIO(`idHorario`).
- `decision` pertenece al dominio enumerado y es obligatoria.
- `fechaRevision` es obligatoria.

7.2.20. SOLICITUD_CAMBIO

- Clave primaria: `idSolicitud`.
- Claves foráneas:
  - `idEstudiante` referencia ESTUDIANTE(`idEstudiante`).
  - `idHorarioOriginal` referencia HORARIO(`idHorario`).
  - Si `idHorarioNuevo` tiene valor, referencia HORARIO(`idHorario`).
  - Si `idCoordinadorApoyo` tiene valor, referencia COORDINADOR(`idCoordinador`).
  - Si `idCoordinadorResolutor` tiene valor, referencia COORDINADOR(`idCoordinador`).
- `estadoSolicitud` pertenece al dominio enumerado y es obligatorio.
- `fechaSolicitud` y `motivo` son obligatorios.
- Una solicitud pendiente no debe tener fecha de resolución. Una solicitud resuelta debe registrar fecha de resolución y resultado.

7.2.21. ESQUEMA_LLAVE

- Clave primaria:

$$(idPersona, idCoordinador, fechaInicio, fecha, fechaAsignacion, idBloque, idValidacion, idRevision, idSolicitud).$$

- Todos los atributos de la clave son obligatorios.
- Las claves foráneas que correspondan deben referenciar las relaciones base del esquema.

7.3. Restricciones de integridad entre relaciones

RII01.
Un coordinador solo puede generar un horario para un programa si existe una autorización vigente para ese programa, con `puedeGenerar` verdadero, en la fecha de generación.

RII02.
Un coordinador solo puede validar un horario si existe una autorización vigente para el programa del horario, con `puedeValidar` verdadero, en la fecha de validación.

RII03.
Un bloque asociado a un programa debe corresponder a una asignatura perteneciente a dicho programa.

RII04.
No puede haber dos imparticiones en la misma aula, fecha e intervalos de tiempo solapados. Los intervalos contiguos, en los que una impartición termina exactamente cuando comienza la siguiente, no se consideran solapados.

RII05.
Solo puede registrarse asistencia para una impartición existente y para un estudiante con una asignación vigente al horario en la fecha de impartición.

RII06.
Una solicitud de cambio solo puede realizarse sobre un horario asignado al estudiante que presenta la solicitud en la fecha de solicitud.

RII07.
Al aprobarse una solicitud de cambio, la asignación del horario anterior debe cerrarse y debe crearse una nueva asignación para el horario resultante. Ambas operaciones deben realizarse en una misma transacción.

RII08.
Un estudiante no puede tener asignaciones vigentes cuyos períodos de vigencia se solapen para horarios distintos del mismo programa y período académico. La comprobación debe consultar el programa y período académico de HORARIO y las fechas de vigencia de HORARIO_ESTUDIANTE. Esta regla debe implementarse mediante un disparador o una validación transaccional que controle también las operaciones concurrentes.

7.4. Integración con el almacén analítico

El almacén ClickHouse es independiente de las relaciones transaccionales definidas arriba; por ello, las claves foráneas de PostgreSQL no se extienden directamente a ClickHouse. El proceso periódico de integración debe conservar la correspondencia con las claves de origen y evitar duplicar registros al reprocesar un período. Las validaciones de carga, los errores de sincronización y la fecha de última actualización deben controlarse en el proceso ETL o en la capa de integración.

Si la capacidad de AULA se utiliza como aforo máximo, debe verificarse además que la cantidad de estudiantes asignados al horario que ocupa esa aula no supere dicha capacidad.